Technology
Arts Sciences
TH Köln

Pick Up The Ball

Special Aspects of mobile autonomous Systems

Computer Science & Engineering

TH Köln

Team 04

Mert Torun

Lars Zimmermann

Prof. Dr. Chunrong Yuan

Winter term 2021/2022

Abstract

This report describes the results of our project in the course Special Aspects of mobile autonomous Systems in winter term 2021 at TH Köln. The main goal of the project was to develop a software that makes the NAO robot pick up a ball from the ground. The project consisted of two major parts. First, an image from the robot's cameras has to be retrieved. Then, using the OpenCV library, these images have to be processed in order to find the ball within the processed image. Once a ball is placed in the area, which is reachable for the NAO, it will pick the ball up. The pickup motion that is performed to do this is created by interpolating the robot's joint positions between previously acquired intermediate positions.

Keywords: NAO robot, Choreographe, OpenCV, Humanoide robots, NAOqi

Table of contents

Introduction

The NAO humanoid robot is developed by Aldebaran Robotics, a French company that was founded in 2005 and is focused on humanoid robotics (Ref. 1).

Image 1: NAO illustration, trying to reach a ball

This report describes the results of a project to use a NAO 6 robot to pick up a ball from the ground. These results show how object recognition can be used. First, an image from the robot's camera has to be retrieved. This is done using the OpenCV library. This image is then processed to find the ball within the image. If a ball is found, the robot adjusts its head's position, so it won't lose track of the ball. If the ball is placed in an area which is reachable for the NAO, it will pick up the ball with a movement. The motion that is performed to do this is created by interpolating the robot's joint's positions between previously acquired intermediate positions.

The project is part of the course Special Aspects of mobile autonomous Systems at the TH Köln in Winter term 2021/2022.

Structure

The report is structured into four main topics to give readers a structured view onto the project.

- **Introduction**
  Brief introduction into the report and the project itself. Requirement and problem analysis is done in this part.

- **Essentials**
  The fundamentals of the project are shown. This includes a description of the used robot and the major used software products, like Jupyter Notebook or NAOqi.

- **Implementation**
  This is split into two parts. The first one is about image and object recognition and the second one describes the pick-up-motion and its development process.

- **Conclusion**
  Review of the project results and the possibilities of future ideas to do are considered.

Problem Analysis

The goal of the project is to recognize a ball shaped object. Typically, this object is represented by a table tennis ball. This ball shaped object then has to be picked up. This problem can be divided into two major parts.

- First, the ball shaped object has to be properly found and recognized by the executing instance. This will require a camera and some software that performs image recognition algorithms on taken pictures.
- Second, after the ball shaped object is recognized, our executing instance, which is the robot, has to perform a specially defined movement to pick up the ball. In order to perform this movement, the robot has to bend down and pick up the ball with his hands without falling down or letting the ball fall down.

Due to numerous limitations, the solution that is developed during this project will not allow the robot to move towards the ball. Rather, the ball has to be placed in the recognition area of the robot. Therefore, the robot does only pick up the ball if it's placed within this area.

Essentials

This chapter covers the hard- and software used in this project. First, an introduction to the NAO robot is given. Afterwards, multiple software products, including the larger libraries which played a major part in the implementation during this project, are explained.

Nao Robot

The NAO robot which was used for this project, is the NAO which was first introduced in 2006 by Aldebaran Robotics. Aldebaran Robotics was rebranded to SoftBank Robotics afterwards. Since then, six versions of the robot were published to the market. Below, you can see an image of the mentioned NAO robot, which shows a humanoid form.

Image 2: NA, a humanoid-type-like robot

The NAO robot is a bipedal robot, whose appearance is inspired by that of a human. It has a wide variety of sensors, such as multiple microphones and speakers, as well as two cameras placed inside it's head. The first camera is placed on the robot's forehead (top camera), the second one is placed on the robot's chin (bottom one). Since the top camera is unable to record the area in front of the feet, which is the essential area we will cover because this is the reachable area to pick up a ball in front of the NAO without moving, only the bottom camera will be used during this project. To be able to move, the NAO possesses an overall of 26 joints (from which 25 can be moved independent of each other).

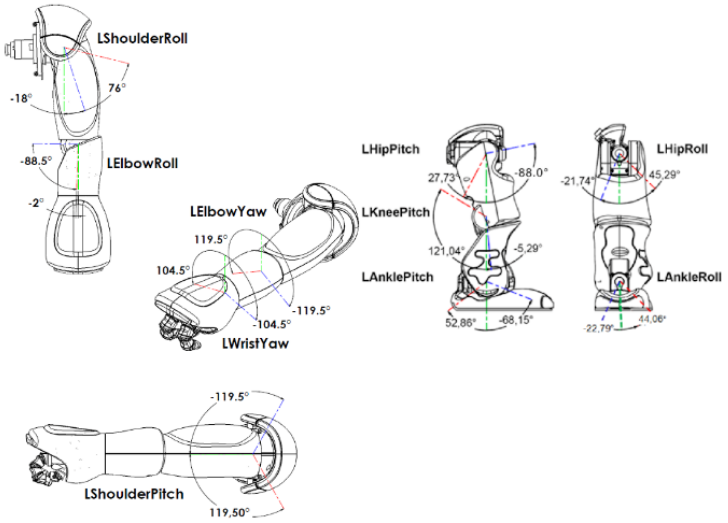

Image 3: Joints in the left arm and left leg of the NAO robot

Most of the robot's joints are placed within its limbs, there are five in each arm as well as five in each leg. On top of that there is one joint to control each hand, two to control the head positions and a single joint to connect the upper body to the robot's legs. The head joints are connected to each other and can't be moved independently - which however doesn't matter for our project's task.

Software

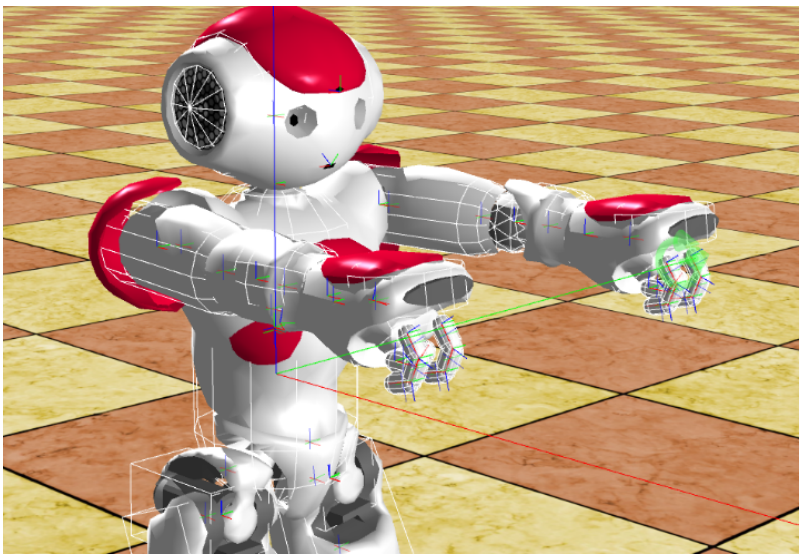

Image 4: NAO pickup motion illustration, step one

The implementation was done using python 2.7.0. In order to develop a precise and functional object detection, the choice fell on the open source library OpenCV. The following subchapters will describe the used libraries throughout the project in detail.

NAOqi

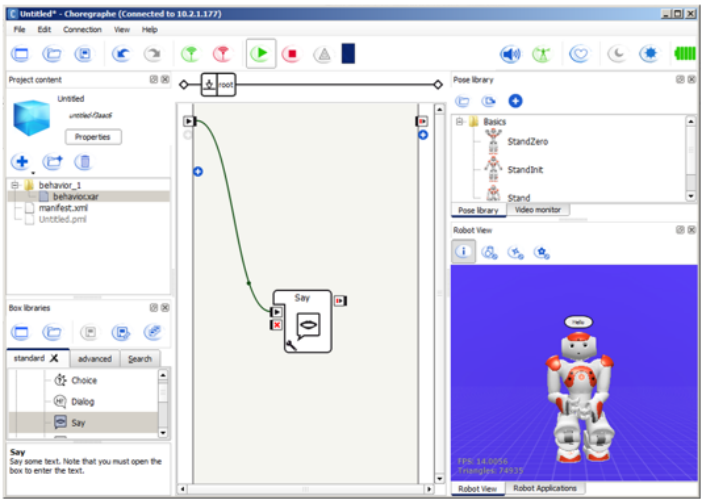

Image 5: Hello World example inside Choreographe - a NAO simulation and run tool

Choreographe supplies us with standardized box elements and a simple GUI which makes "programming" NAO really easy and convenient. These box elements allow an easy creation and modification of tasks, so that the developer gets a sense of how to deal with robot instructions. There is an opportunity to create a simulated NAO robot which executes these "box instructions" in order to test movements or behaviors, but it also supports direct connection to a real NAO as well, which allows it to run the simulated software on a real NAO.

On the other hand, there is the NAOqi Framework which is available for multiple programming languages, which offers a vast more complexity to deal with NAO and allows way more to do. For this project, however, we will only be using the Python API. This API allows communication with a broker running on the NAO robot.

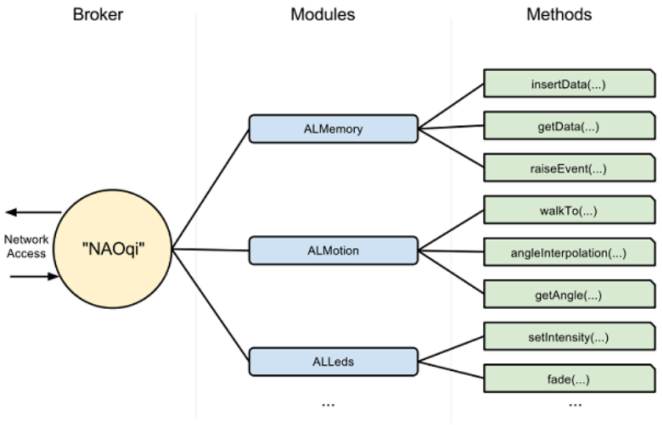

Image 6: Network access through a broker

As you can see in the image above, the broker possesses multiple modules with numerous functions. The NAOqi Python API allows the user to create a proxy and call these modules and their appropriate methods. For this projects only selected modules are being used:

- **ALMemory**: which allows reading parameters from within the robot such as joint positions
- **ALMotion**: which can be used to move the robot's joints
- **ALRobotPosture**: which includes a set of predefined postures into the robot
- **ALVideoDevice**: which grants access to the cameras

OpenCV

Image 7: OpenCV logo

OpenCV is an **O**pen source **C**omputer **V**ision library. In 2006 the company Intel published Version 1.0 to build an infrastructure for computer vision applications. Meanwhile, version 4.2.0 includes more than 2500 optimized algorithms, which can be used to detect and recognize faces, identify objects, classify human actions in videos, track camera movements, track moving objects and much more.

Imutils

Imutils is an open source library provided by Adrian Rosebrock. It offers basic image processing functions such as translation, rotation, resizing, skeletonization, and displaying Matplotlib images.

Python Image Library (Pillow)

The Python Image Library is an open source library for the Python programming language that adds support for opening, manipulating, and saving images and more. It was released in 2009 by Alex Clark.

Implementation

The project is structured into three parts. On the one hand we are going to try and find the ball which is to be picked up, this means: It has to be localized. In order to realize this, one of the NAO's cameras is being used. The whole process of gathering a picture as well as processing it so the ball can be found is described in the subchapter **Image Recognition**. The second part of the project is to pick up the ball as soon as it is in a reachable position for the robot, without moving. Because moving could make the robot fall down, since it can't balance itself because it's missing proper sensors for this - but later, more on this. As soon as the ball is placed in a reachable position, a pickup motion is started. The development and the sequence of this motion is explained in the subchapter **Fetching the images**. The two parts of the project are connected through an event. As soon as the ball is placed in a reachable position, an event is fired. This event then triggers the pickup motion to start.

Image Recognition

As already mentioned beforehand, OpenCV is used to realize ball / object detection. This subtopic is split into three parts. First, the images from the NAO camera have to be retrieved so that the connected hardware can perform image processing steps and recognize the object. There has to be an option to define the target object which the robot will track, since the recognition has to work at different environments properly and therefore has to work with adjustable ball colors. After this is done, the detection algorithm can start. Last, the robot has to keep his focus on the target object by executing some head movements and keep track of it.

Fetching the images

In order to simultaneously query images from both cameras and also perform other actions, the image acquisition runs in two separate threads, one for each of the robots cameras. Although only the bottom camera is used for our project, the pictures from both cameras will be queried to be able to possibly extend the project in the future.

After thread start, the proxy subscribes to the camera event and has to unsubscribe, even if the program is interrupted or terminated. The images are retrieved by the method **getImageRemote()** which returns them in a byte array format. To get them in a more user-friendly format, the camera control thread converts them to PIL format. After converting the image, an event is fired that triggers the image recognition method so the object can be detected.

Object detection

First, the method of **detection** creates a mask which plays a decisive role in the recognition part. The mask blurs the frame to reduce high frequency noise and allows an improved focus on the structural objects inside the frame. The recognition in this project is done by shape analysis. Shapes that have the same color or intensity as a predefined color are detected. The color has to be defined in the **HSV** - **H**ue **S**aturation **V**alue - color space. This definition of color has the advantage of considering the saturation and brightness as the value of a color. This way it is possible to consider difficult environmental conditions such as bright ambient light which, depending on the material properties of the ball, leads to a stronger reflection. During development, the test ball was orange so that a lower and an upper boundary had to be defined for this orange.

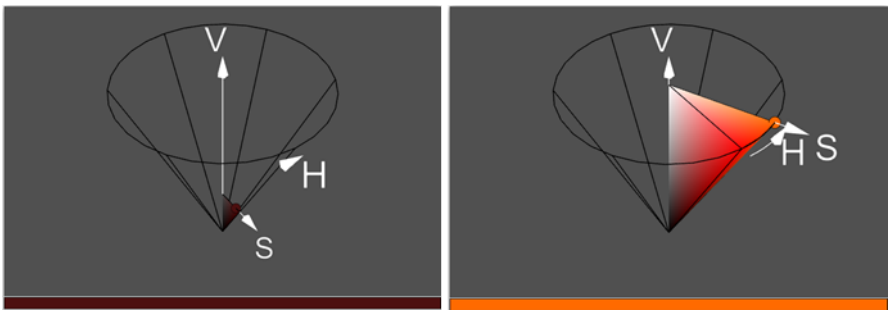

Image 8: Lower and upper boundaries of the predefined color "orange" in HSV-Color space

With assistance of OpenCV the mask can be edited so that only contours with the predefined color are visible. The remaining area can now be analyzed to see if it is a ball or not. To filter out misc or fail detections of objects which were falsely recognized, only the position of a circular area is determined if it has a predefined radius size.

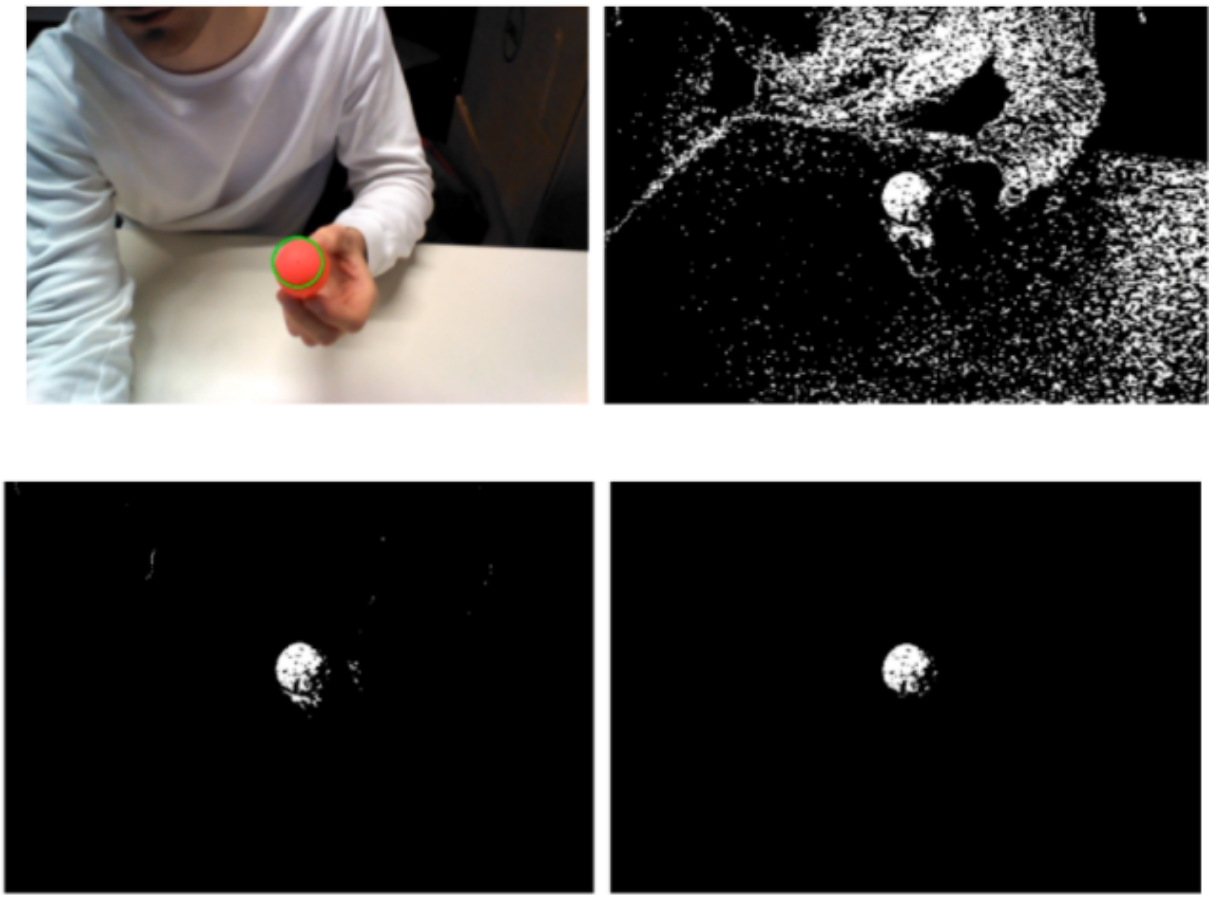

Image 9: Detected area in the mask using a ball step-wise

In order to improve the identification, which area the algorithm has detected, a circle is drawn around it.

Target initialization

As described above, the environment has a significant influence on object detection. In order to recognize objects successfully in environments other than the test environment, an initialization mode was implemented. The user can trigger it by pressing the **I** key on the keyboard. The detection mode stops and three windows will open. One window shows the original image of the robot and another shows the created mask as shown in Image 9. Also, a configuration window, as shown in Image 10, will be opened.

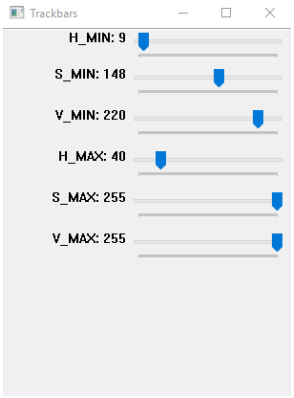

Image 10: Configuration trackbar in initialize mode

The user gets the opportunity to find the lower and upper boundaries of the color in the HSV color space which is suitable for the objects which will be detected. To quit the initialization mode, press **k** on the keyboard.

Target tracking

After the object is detected and localized, the robot has to follow the object's position by some head movements in order to keep the ball within the receiving field. To achieve this, the robot tries to focus the object in the center of the image. That means, if the object moves out of the center, the head angles of the robot will be adjusted so that the ball is centered in the image again. The correction of the angles is achieved with the help of a PD Controller. If now the ball reaches a predefined position, an event will be fired which triggers the pickup motion.

Pick Up Motion

The NAOqi library provides a wide range of functions including multiple pre implemented poses which the robot can enter, like sitting etc. There is, however, neither a motion to pick something up from the ground in front of it nor any position that would allow the NAO to reach the ground with its tiny hands without falling down. Please note that NAO only has three fingers, which makes grabbing already difficult, and you should aim to grab an object with both hands at the same time - so the object won't fall down! However, since NAOqi doesn't offer any proper pose we could use, therefore a new position has to be found. The easiest way to create a new movement for the NAo is to use the animation mode from choreographe. Using this mode, it is possible to create a movement by defining multiple intermediate positions. After moving the NAO into the desired position, all joint positions can be read out. This is done by interpolating the joint positions from one desired intermediate position to the next one, a movement can be created. The full movement is described in the following.

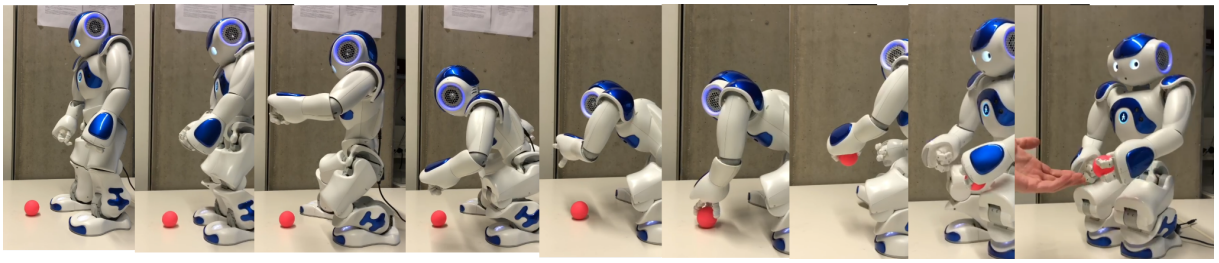

Image 11: Pick-up motion, starting from left to right

As you can see in image 11, we are starting from a standing position, the robot then prepares to bend down and pick up the ball by bending its knees and widening its stance, so its stand is more stable. The NAO then starts to bend over and positions its hand above the ball. On top of that, the hand joint is opened, so the ball can be grabbed (Image 10 Sub 5, from left to right, the 5th image). Then NAO grabs the ball. At first, the hand is lowered, so the ball is in the right place to be grabbed. In the following position, the hand is closed, so the ball can be picked up. The robot then starts getting up by slowly moving its upper body in an upright position. On top of that, the feet are moved closer to each other, so the robot can get up in the following steps. Then the robot starts getting up by moving its center between its feet. Now we are in crouch posture.

Conclusion

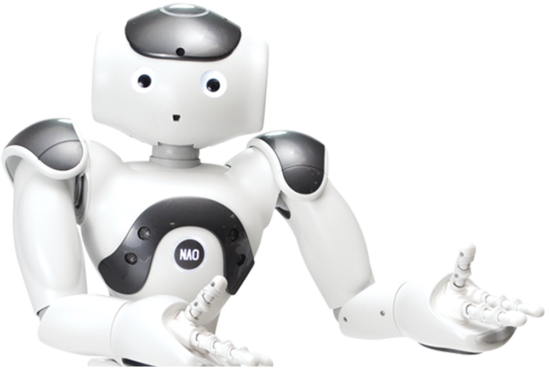

Image 12: NAO

In this final chapter of the report, the whole project is summarized and finalized. At first, the project is reviewed and evaluated. Afterwards, problems and their consequences on the project are discussed in detail. In the end, some ideas are given how similar projects in future may solve these problems and how this project could possibly be extended.

Review

The main goal of the project was to find a ball shaped object using a NAO robot's camera and to pick it up, so the robot can perform further actions with this ball in his hands. This goal can be achieved in a limited manner. While the basic functions, like recognizing the ball and picking it up, work fine, the robot is not very flexible doing this. Since, the starting position is fixed. To find the ball it has to be moved into the robot's FOV (Field of View). To be able to pick up the ball, it has to be placed in a small area, which the NAO robot can reach from its current position. In an extension of the project it is certainly possible to solve these issues, therefore the project can be seen as a good initial attempt to solve the problem of finding and picking up objects using a NAO robot.

Problems

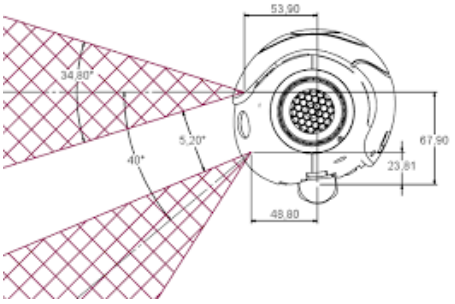

Image 13: NAO head camera viewing angles

During our project, numerous problems occurred, which were not known prior. This was due to limitations on the software and hardware side of the NAO robot. The first issue comes up during the image acquisition process. Since the whole image processing code runs on a remote computer instead of the robot, the images have to be transmitted from the robot to the controlling PC. Unfortunately, this transmission is very slow, causing a frame rate of only 1 fps, when using a wireless connection (which we did). This frame rate increases to only(!) 2 - 4 fps when using a wired connection - which seems very slow, however. Another major issue is caused by the design of the NAO robot. Since the two cameras the robot has are placed on it's top and bottom head (forehead top, chin bottom), the overlapping field of view is very small. Therefore, using both cameras to estimate the distance to the ball is complicated, if not even impossible. Especially since the area on the floor, which is the reachable area of NAO, can never be seen by the top camera. The third major concern is that there are no sensors in NAO that allow it to hold the balance on its own, like a gyroscope. Because of this, the pick up motion can not be adapted since a slightly different behavior could cause the robot to lose its balance and this cannot be detected, which would cause the robot to fall. The missing ability to measure the balance means that the robot cannot simply be used on uneven grounds.

Future Work

Image 14: NAO future

After we finished the project, it was obvious that there are still many parts of the original problem left unsolved. As explained already beforehand, with the current solution the robot is limited to only find the ball within its field of view and can only pick up objects from a certain area which it can reach from its current position. Therefore, the most obvious addition to our current solution would be to explicitly search for the ball and if it's not currently seen to adjust its own position if the ball is not in a reachable area. This would make the robot more autonomous in the search and pick up process, instead of having a predefined starting position. On top of that, it is possible to improve the current solution approach without adding new features. For example, it is possible to execute the code on the robot's processing unit instead of an external controlling PC, which processes the images. This would most probably increase the frame rate and improve the image recognition process.

Sources

Ref. 1 | https://www.ithistory.org/db/companies/aldebaran-robotics | Accessed: 07.11.2021

Ref. 2 | Siegwart, R. et. al., Introduction to Autonomous Mobile Robots, MIT Press, 2010.

Ref. 3 | OpenCV, https://opencv.org | Accessed: 10.11.2021

Ref. 4 | Choregraphe Suite, http://doc.aldebaran.com/2-1/software/choregraphe/index.html | Accessed: 30.11.2021

Ref. 5 | NAOqi - Developer guide, http://doc.aldebaran.com/2-1/index_dev_guide.html | Accessed: 30.11.2021

Ref. 6 | NAO, https://developer.softbankrobotics.com | Accessed: 30.11.2021

Ref. 7 | Learning to grab objects with the NAO robot, https://fse.studenttheses.ub.rug.nl/10695/1/Bachelorverslag.pdf | Accessed: 30.11.2021

Ref. 8 | Student project: Do it NAO, https://youtu.be/neqwbiSGxoE | Accessed: 30.11.2021